



Capstone Track 2: Digital Twin with Omniverse Integration



Objective:

Design and deploy a real-time **AI-integrated digital twin** using **NVIDIA Omniverse**, with AI inference served via **Triton**, optimized using **TensorRT**, and optionally deployed at the **edge (Jetson)** or **cloud (DGX, EC2, etc.)**.

You'll simulate a live environment (factory floor, warehouse, robot task, or smart building) and connect AI-driven perception to the simulation using **Omniverse Isaac Sim** or **Omniverse Replicator**.



Step-by-Step Instructions

Step 1: Choose Your Scenario

Select a digital twin use case:

- 🤖 Factory robot arm picking objects
- 🏭 Conveyor belt anomaly detection
- 🏢 Smart building with foot traffic + HVAC modeling
- 📦 Warehouse drone coordination

→ Define scenario input/output:

- Sensors → AI Model → Response → Simulation Feedback
-

Step 2: Set Up Omniverse Environment

Install Omniverse components:

1. [Download Omniverse Launcher](#)
 2. Install:
 - **Omniverse Isaac Sim** (for robotics)
 - **Omniverse Code** (for Python scripting & digital twin logic)
 - **Nucleus Server** (for asset storage)
-

Step 3: Build the Digital Twin World

In **Isaac Sim** or **Omniverse Code**:

- Import 3D assets (robots, sensors, environment)

- Define sensor positions: camera, lidar, proximity
- Simulate physical movement and time
- Add scripting for event triggers (e.g., person enters zone)

Use USD (Universal Scene Description) for asset interop.

Step 4: Integrate Real-Time AI Inference via Triton

Use Python APIs or ROS bridge to send data from the Omniverse sensor to a Triton model:

1. Capture simulated sensor data (e.g., camera image)
2. Preprocess and send via gRPC:

```
import tritonclient.grpc as grpcclient
client = grpcclient.InferenceServerClient(url="localhost:8001")
# Create request with sensor input and send to model
```

3. Receive inference output (e.g., object detection or segmentation)
 4. Use output to trigger logic inside the simulation
-

Step 5: Optimize the Model with TensorRT

Use TAO Toolkit or export to ONNX, then convert:

```
trtexec --onnx=model.onnx --saveEngine=model.engine --fp16
```

→ Place TensorRT model in `models/<model_name>/1/` with config for Triton.

Step 6: Containerize AI + Simulation Logic

Create two containers:

- 🎮 **Omniverse Container:** With Isaac Sim or Omniverse Code
- 🧠 **Triton AI Server:** Running optimized AI model

Use Docker Compose or K8s to orchestrate:

```
services:
  omniverse_sim:
    image: omniverse/isaac-sim:latest
    ...
  triton:
    image: nvcr.io/nvidia/tritonserver:24.03-py3
    ...
```

Step 7: Cloud or Edge Deployment

Cloud:

- Use AWS EC2 (g5, g4dn) or Azure NC-series
- Or deploy on **DGX Cloud** for large-scale sims

Edge:

- Run Isaac Sim on host, deploy inference to **Jetson Orin/Xavier**
 - Use ROS2 or MQTT bridge for async data
-

Step 8: Add Monitoring & Logging

- Log model inputs/outputs for debugging
 - Visualize simulation + AI decisions in UI
 - Optionally log to Kafka, MQTT, or CSV for analysis
-

Step 9: Deliverables

-  **Project Report**
 - Use case overview
 - Simulation architecture
 - Inference integration
 - Screenshots + diagrams
 -  **Deployment Pipeline**
 - Docker Compose/K8s config
 - Model repo
 - Isaac Sim scripts
 -  **Demo Video**
 - Screen capture of simulation
 - Live inference loop
 - (Optional) Side-by-side: Omniverse + Jetson output
-